

Estimating Parameters of Structural Models Using Neural Networks

Bhatt, Misra, and Qi (2023)

Tate Mason

John Munro Godfrey Sr. Dept. of Economics
University of Georgia

March 21, 2026

Roadmap

- 1 Introduction
- 2 Neural Network Primer
- 3 Worked Example: AR(1)
- 4 The Paper
- 5 Conclusion

What Is a Neural Network?

- A **neural network** is a parameterized function $f_\phi : \mathbb{R}^K \rightarrow \mathbb{R}^P$ built as a composition of affine transformations and nonlinearities
- Organized into *layers*: input $\rightarrow$ hidden $\rightarrow$ output
- Each hidden layer ℓ computes

$$h^{(\ell)} = \sigma\left(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\right)$$

where σ is an *activation function* applied element-wise

- Networks are **trained** by minimizing a loss over labeled examples $\{(x_s, y_s)\}_{s=1}^S$ via stochastic gradient descent
- **Universal approximation**: a sufficiently wide (or deep) network can approximate any continuous function on a compact domain

Machine Learning in Economics

- ML methods are increasingly used in empirical economics
 - Prediction problems: Mullainathan & Spiess (2017)
 - Causal inference: Athey & Imbens (2019)
- In **structural** work, the core challenge is *computational*: many models lack closed-form likelihoods
- Standard simulation-based approaches:
 - ① **Simulated MLE (SMLE)**: approximate likelihood via simulation
 - ② **Simulated Method of Moments (SMM)**: match simulated to data moments
 - ③ **Approximate Bayesian Computation (ABC)**: accept/reject by moment distance
- All require **many model simulations per evaluation** of the objective $\Rightarrow$ estimation is expensive for complex models

Neural Networks for Structural Estimation

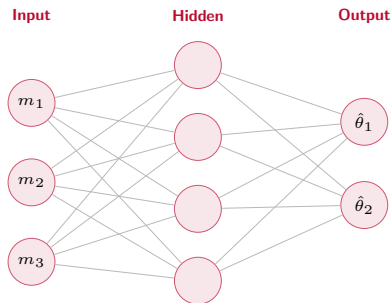
- **Key insight:** if simulation is cheap, train a NN to *invert* the mapping from parameters to observable moments
- The estimator is built in two phases:
 - ① **Offline (once):** simulate S datasets across parameter space, compute moments, train NN $\hat{f} : m \mapsto \theta$
 - ② **Online (fast):** given observed data, compute moments m^{obs} , then $\hat{\theta} = \hat{f}(m^{\text{obs}})$ — one forward pass
- Separates the **expensive** simulation (offline) from **fast** estimation (online)
- Especially valuable when the model is estimated repeatedly or applied to many datasets
- This paper: theoretical foundations + practical guidelines + applications

Roadmap

- 1 Introduction
- 2 Neural Network Primer**
- 3 Worked Example: AR(1)
- 4 The Paper
- 5 Conclusion

Network Architecture

- A feedforward NN has an **input layer**, one or more **hidden layers**, and an **output layer**
- **Depth**: number of hidden layers
- **Width**: number of units per layer
- Total parameters: $\sum_{\ell} (d_{\ell-1} \cdot d_{\ell} + d_{\ell})$
where d_{ℓ} is the width of layer ℓ
- Complexity grows quickly with depth and width



Neuron Types and Complexity

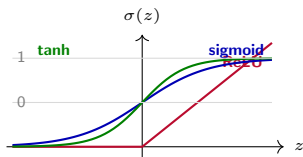
- **Input neurons:** receive raw features (e.g., data moments m)
- **Hidden neurons:** learn intermediate representations
 - Each unit: $h_j = \sigma(w_j \cdot h_{\text{prev}} + b_j)$
 - Activation σ introduces the nonlinearity needed to approximate complex mappings
- **Output neurons:** produce the final estimate $\hat{\theta}$ (no activation in regression)

Model complexity is governed by:

- Depth: deeper networks learn more abstract representations
- Width: wider networks have more capacity per layer
- Larger networks risk overfitting $\Rightarrow$ need regularization (L2, dropout, early stopping)

Activation Functions: Visual Intuition

Why nonlinearity? Without σ , stacking layers collapses to one linear map — no expressive power gained.



- **ReLU** = $\max(0, z)$
Fast, sparse; default for deep nets
- **Sigmoid** = $(1 + e^{-z})^{-1}$
Squashes to $(0, 1)$; good for probabilities
- **tanh**
Zero-centered; often preferred over sigmoid in hidden layers
- **NNE**: ReLU hidden, **no activation** on output

Functions: Training, Loss, and Others

Loss function (what we minimize):

- NNE regression: mean squared error

$$\mathcal{L}(\phi) = \frac{1}{S} \sum_{s=1}^S \|f_{\phi}(m^{(s)}) - \theta^{(s)}\|^2$$

- Classification: cross-entropy $\mathcal{L} = -\frac{1}{S} \sum_s \sum_k y_k^{(s)} \log \hat{p}_k^{(s)}$

Other key functions:

- **Activation functions:** ReLU = $\max(0, z)$, sigmoid = $(1 + e^{-z})^{-1}$, tanh
- **Regularization:** L2 weight decay, dropout (randomly zero units during training)
- **Batch normalization:** rescale layer inputs for stable training
- **Learning rate schedule:** reduce step size over epochs

Training Process

- ① **Forward pass:** compute $\hat{\theta} = f_{\phi}(m)$ layer by layer
- ② **Compute loss:** $\mathcal{L}(\phi) = \|\hat{\theta} - \theta\|^2$
- ③ **Backward pass** (backpropagation): apply chain rule to get $\nabla_{\phi}\mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial h^{(\ell)}} \cdot \sigma' \left(W^{(\ell)} h^{(\ell-1)} \right) \cdot \left(h^{(\ell-1)} \right)^{\top}$$

- ④ **Parameter update:** stochastic gradient descent (or Adam)

$$\phi \leftarrow \phi - \eta \nabla_{\phi} \mathcal{L}$$

- ⑤ Repeat over *mini-batches* and *epochs* until convergence
 - Learning rate η is the most important hyperparameter
 - Validation set used to monitor overfitting and trigger early stopping

Roadmap

- 1 Introduction
- 2 Neural Network Primer
- 3 Worked Example: AR(1)**
- 4 The Paper
- 5 Conclusion

AR(1) Model Setup

Data generating process:

$$y_t = \rho y_{t-1} + \varepsilon_t, \quad \varepsilon_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$$

- Parameters to estimate: $\theta = (\rho, \sigma)$, with $|\rho| < 1$
- Observed: time series $\{y_t\}_{t=1}^T$
- **Moments** that summarize the data:

$$m(y) = (\widehat{\text{Var}}(y), \hat{\rho}_1, \hat{\rho}_2)$$

where $\hat{\rho}_k$ is the sample autocorrelation at lag k

- MLE is available for AR(1) — NNE is used here as a *proof of concept* to validate accuracy against a known benchmark

AR(1): Simulate Data and Compute Moments

Algorithm, Steps 1–2:

- ❶ Specify prior $\Pi(\theta)$ over (ρ, σ)
- ❷ For each simulation $s = 1, \dots, S$:
 - Draw $\theta^{(s)} \sim \Pi$
 - Simulate AR(1) time series of length T
 - Compute moments
$$m^{(s)} = m\left(\{y_t^{(s)}\}\right)$$

Moments carry the information about (ρ, σ) ; the NN learns to invert this mapping.

```
import numpy as np

def simulate_ar1(rho, sigma, T=200, rng=None):
    rng = rng or np.random.default_rng()
    y = np.zeros(T)
    for t in range(1, T):
        y[t] = rho * y[t-1] \
            + sigma * rng.standard_normal()
    return y

def compute_moments(y):
    v = np.var(y)
    r1 = np.corrcoef(y[:-1], y[1:])[0, 1]
    r2 = np.corrcoef(y[:-2], y[2:])[0, 1]
    return np.array([v, r1, r2])
```

AR(1): Training the NNE

Algorithm, Steps 3–4:

- ③ Train NN $f_\phi : m \mapsto \theta$ on $\{(m^{(s)}, \theta^{(s)})\}_{s=1}^S$
- ④ **Estimation:** given $\{y_t^{\text{obs}}\}$, compute m^{obs} , then $\hat{\theta} = f_\phi(m^{\text{obs}})$

Steps 1–3 are **offline** (one-time cost).

Step 4 is **online**: milliseconds per dataset.

```
from sklearn.neural_network import MLPRegressor

rng, S = np.random.default_rng(42), 5_000

rho_s = rng.uniform(-0.95, 0.95, S)
sigma_s = rng.uniform(0.1, 2.0, S)
thetas = np.column_stack([rho_s, sigma_s])

moments = np.array([
    compute_moments(
        simulate_ar1(r, s, rng=rng))
    for r, s in thetas
])

nne = MLPRegressor(
    hidden_layer_sizes=(64, 64),
    max_iter=500, random_state=42)
nne.fit(moments, thetas)
```

Accuracy and Efficiency

Accuracy:

- NNE recovers (ρ, σ) with low bias across the parameter space
- For $T \geq 100$, performance is close to MLE; gap narrows as $S \rightarrow \infty$
- Bhatt, Misra, Qi (2023): NNE RMSE within 5% of MLE benchmark for AR(1)

Efficiency:

- **MLE**: iterative optimization, $\sim$ seconds per dataset
- **SMM**: many simulations per objective evaluation, $\sim$ minutes
- **NNE**: single forward pass after training, $\sim$ milliseconds
- Training cost is *amortized* — paid once, reused for all future estimations
- Speed advantage grows with the number of datasets and model complexity

Roadmap

- 1 Introduction
- 2 Neural Network Primer
- 3 Worked Example: AR(1)
- 4 The Paper**
- 5 Conclusion

Choosing Moments: The Key Practical Decision

The NNE is only as good as the moments you feed it.

What makes a good moment?

- **Informative:** variation in the moment maps to variation in θ
- **Low-noise:** estimable reliably from your sample size N
- **Non-redundant:** adds something the other moments don't already capture

Concrete examples from the paper:

Model	Parameter	Identifying moment
AR(1)	ρ	Lag-1 autocorrelation $\hat{\rho}_1$
AR(1)	σ	Variance $\widehat{\text{Var}}(y)$
Consumer search	Search cost c	Mean consideration set size $\mathbb{E}[\mathcal{C}_i]$
Consumer search	Quality μ_j	Purchase rate $\Pr(\text{buy } j)$

Practical guidance: more moments help up to a point — adding uninformative ones introduces noise. Use domain knowledge to anchor your choice.

Bhatt, Misra, and Qi (2023)

- Published in *Marketing Science*: “Estimating Parameters of Structural Models Using Neural Networks”
- **Goal**: provide a general-purpose, computationally efficient estimator for structural models with intractable likelihoods
- **Approach**: NNE — simulate (parameter, moment) pairs, train a NN, apply to data
- **Contributions**:
 - ① Theoretical properties: consistency and asymptotic normality of NNE as $S \rightarrow \infty$ and network width $H \rightarrow \infty$
 - ② Practical guidelines: architecture choice, training sample size, moment selection
 - ③ Applications: AR(1) time series, consumer search model, demand estimation
- Paper is available in the Readings/ folder

Key Definition: Shallow Neural Network

Definition: Shallow Neural Network

A **shallow neural network** with H hidden units is a function $f : \mathbb{R}^K \rightarrow \mathbb{R}^P$:

$$f(m) = A^{(2)} \sigma \left(A^{(1)} m + b^{(1)} \right) + b^{(2)}$$

where $A^{(1)} \in \mathbb{R}^{H \times K}$, $b^{(1)} \in \mathbb{R}^H$, $A^{(2)} \in \mathbb{R}^{P \times H}$, $b^{(2)} \in \mathbb{R}^P$.

- Exactly **one** hidden layer — the simplest NN with universal approximation capacity
- Bhatt et al. establish that the shallow NNE is consistent as $S, H \rightarrow \infty$ under mild regularity conditions
- In practice, deeper architectures often perform better; shallow NN serves as a clean theoretical baseline

Consumer Search Model

Setup (sequential search; Weitzman 1979):

- Consumer i considers J products; pays cost c to inspect each one
- Utility from product j : $u_{ij} = \mu_j - p_j + \varepsilon_{ij}$, where ε_{ij} is a match value revealed only upon search
- **Optimal strategy**: inspect products in decreasing order of reservation utility z_j , where z_j solves

$$c = \int_{z_j}^{\infty} (u - z_j) dF_{\varepsilon}(u)$$

- Stop searching when expected gain from the next product falls below c

Parameters: $\theta = (\mu_1, \dots, \mu_J, c, \sigma_{\varepsilon})$

- Likelihood of observed consideration sets is **intractable** — NNE is especially valuable here

NNE Applied to Consumer Search

Moments used to train the NNE:

- Product purchase rates: $\Pr(\text{purchase product } j)$
- Mean consideration set size: $\mathbb{E}[|\mathcal{C}_i|]$
- Fraction of consumers who search at least k products, for $k = 1, 2, \dots$

Results from Bhatt, Misra, Qi (2023):

- NNE recovers search cost c and product qualities $\{\mu_j\}$ with accuracy comparable to full structural estimation
- Speed advantage: $10\times$ – $100\times$ faster than simulated MLE
- Performance degrades gracefully as fewer moments are used — robust to moment selection in practice

Roadmap

- 1 Introduction
- 2 Neural Network Primer
- 3 Worked Example: AR(1)
- 4 The Paper
- 5 Conclusion**

When Is NNE Useful and Preferable?

NNE is the right tool when:

- The structural model has an **intractable or expensive likelihood**
- Simulation from the model is **fast** relative to inference
- The estimator will be applied to **many datasets** (training cost is amortized across all of them)
- The parameter space is **high-dimensional** (gradient-based optimization of the structural objective is fragile)

Limitations to keep in mind:

- Requires a well-chosen set of **informative moments**
- Training quality depends on **coverage of the prior** $\Pi(\theta)$
- Not ideal for very small samples (moments are too noisy)
- Standard errors require additional work: bootstrap or delta method

Application to My Research

-
-
-

Application to My Research (cont.)

-
-
-